ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет информатики, математики и компьютерных наук

Программа подготовки бакалавров по направлению

01.03.02 Прикладная математика и информатика

Тюрин Владислав Николаевич

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Построение графа знаний на основе пользовательских данных с возможностью экспорта в Obsidian

Научный руководитель

Приглашенный преподаватель

_____

В. М. Кочеганов

Нижний Новгород, 2026

ОГЛАВЛЕНИЕ

Введение 4

1. Теоретическая часть 4
   1.1. Системы управления персональными знаниями 4
   1.2. Технологии поиска и обработки естественного языка в системах RAG 4
   1.3. Анализ существующих решений 4
   1.4. Методология оценки качества 4
      1.4.1 Фреймворки 4
      1.4.2 Метрики 4

2. Дизайн алгоритмов 4
   2.1. Архитектура алгоритма построения графа знаний 4
   2.2. Алгоритм GraphRAG 12

3. Сбор и подготовка данных 16
   3.1. Требования к данным 16
   3.2. Поиск данных 17
   3.3. Создание валидационного датасета 19
      3.3.1 Алгоритм генерации связей 20
      3.3.2 Алгоритм извлечения контекста 21
      3.3.3 Алгоритм LLM-классификации отношений 23

4. Эксперименты 24
   4.1. Оценка конвейера построения графа знаний 24
      4.1.1 Извлечение кандидатных пар 24
      4.1.2 Классификация типов связей 28
   4.2. Оценка конвейера GraphRAG 30
      4.2.1 Референсное решение (ObsidianRAG) 31
      4.2.2 Влияние промпта и языковой модели 32

4.3. Итоговое сравнение и ограничения исследования 36

5. Заключение 36

6. Список литературы 36

7. Приложение 36

ВВЕДЕНИЕ

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Системы управления персональными знаниями

1.2 Технологии поиска и обработки естественного языка в системах RAG

1.3 Анализ существующих решений

1.4 Методология оценки качества

1.4.1 Фреймворки

1.4.2 Метрики

ДИЗАЙН АЛГОРИТМОВ

2.1 Архитектура алгоритма построения графа знаний

Разработанная система является конвейером для автоматического нахождения, типизации и инкрементального сохранения семантических связей между заметками (Markdown-файлами) в хранилище Obsidian. Процесс построения состоит из нескольких шагов: от сканирования хранилища до записи найденных отношений в файлы. Алгоритм поддреживает сохранение состояния файлов в директории и контрольные точки (checkpoints), которые позволяют алгоритму продолжить работу с момента прерывания и нацелены на сохранение вычислительных ресурсов.

Общая схема конвейера представлена на рисунке 1.

Рис. 1 – Общая схема конвейера построения графа знаний

Сканирование хранилища и обнаружение изменений (VaultManager)

На этом этапе модуль `VaultManager` рекурсивно обходит директорию хранилища, формируя список всех `markdown`-файлов. Для каждого файла вычисляется SHA-256 хеш-сумма его содержимого для иденификации изменений. Полученный хэш сравнивается с сохраненным в метаданных `metadata.json`, с предыдущего запуска. В результате файлы разбиваются на три категории: новые (отсутствующие в метаданных), изменённые (не совпадает хеш) и удалённые (есть в метаданных, но отсутствует на диске). Если метаданных нет, то все файлы считаются новыми.

Данный подход позволяет экономить вычислительные ресурны, поскольку позволяет обрабатывать только измененные файлы, а не все хранилище целиком.

Этап 2. Разбивка на чанки и индексация (VectorStore / LanceDB). Каждый новый или измененный файл проходит процесс индексации. Эта процедура состоит из двух частей: обработки содержимого заметок и подготовки индексов.

Изначально текст каждого Markdown-файла состоит из двух частей: основного содержимого и блока ссылок, разделенными отделенным заголовком `link_header`. Индексируется только основной текст заметки.

Текстовый индекс строится на основе лемматизированного содержимого файлов. Лемматизация проводится при помощи модели `ru_core_news_sm` или `en_core_web_sm` из `spaCy`. Из текстов извлекаются леммы всех алфавитных и числовых токенов в нижнем регистре. Преобразованные тексты передаются в LanceDB, где на них строится FTS-индекс, использующий алгоритм BM25. Данный подход позволяет увеличить точность поиска, нивелируя проблему, связанную с изменением форм слов, в которых, например, могут быть изменены окончания или беглые гласные в корне, что особенно распространено в русском языке. Стоит отметить, что вместе с преобразованным текстом сохраняется и оригинальный.

Построение векторного индекса начинается с разбиения содержимого файла на перекрывающиеся чанки. Поддерживается две стартегии разбиения:

- *Рекурсивная посимвольная* (`RecursiveCharacterTextSplitter`) из LangChain. Она разбивает текст иерархически по заданным разделителям, расположенным в порядке их приоритета, ((`\n#` , `\n##` , `\n\n`, `.`, `\n` и т.д.)), при этом не превышая заданный размер чанка (по умолчанию 3,000 символов). Данная стратегия сохраняет целостность предложений и структуру Markdown-файлов, что крайне важно для вычисления эмбеддингов и формирования контекста для LLM.

- *Оконная по предложениям* (`SentenceWindowSplitter`), реализованная с помощью spaCy. Данная стратегия разбивает текст по предложениям с помощью spaCy (`doc.sents`) и формирует фрагменты из каждого предложения с заданным окном из соседних предложений (`window_before` = 1, `window_after` = 1 по умолчанию). Такой подход нацелен на максимизацию точности формирования контекста и его полноты без потери семантики, что достигается путем сохранения целостности предложений. Главным недостатком является большое перекрытие чанков: даже при окне в одно предыдущее и следующее предложение, каждое предложение используется в трех чанках.

Далее для каждого чанка следует вычисление плотного векторного эмбеддинга при помощи модели из Sentence-Transformers, например популярной мультиязычной модели `BAAI/bge-m3`.

Полученные индексы объединяются друг с другом, а также с метаданным в виде пути к файлу и хеша, после чего сохраняются в базу данных. Хранение обоих форматов данных в одной таблице позволяет использовать гибридный поиск. LanceDB позволяет регулировать баланс между векторным и полнотекстовым поиском при помощи параметра `vector_weight` $\in [0, 1]$: при `vector_weight` $= 1.0$ используется только ANN-поиск, при `vector_weight` $= 0.0$ — только BM25, промежуточные значения используют оба поиска, сортируя результаты по унифицированной оценке релевантности.

Результатом поиска является *top-k* подходящих фрагментов текста.

Этап 3. Гибридный поиск пар-кандидатов (Retrieval). На данном этапе для каждого чанка выполняется поиск семантически близких чанков из других файлов. Доступно три стратегии поиска:

- *broad*, основанная на векторном поиске. Для каждого чанка производится поиск k ближайших соседей из других файлов. Поддреживается два режима поиска

– *chunk*, при котором поиск происходит для каждого чанка в файле

– *title_summary*, при котором поиск ведется только на основе названия файла и его краткого содержания (первого абзаца)

Второй подход позволяет искать схожие чанки на основе семантики всего файла, что значительно сократить уровень шума и ускоряет процесс, но может не найти часть релевантных фрагментов.

- *strict*. Она основана на поиске прямых упоминаний сущностей в текстах заметок. Поиск производится по лемматизированному тексту, что решает проблему словоизменения, несовпадения регистров и использования специальных символов, таких как дефис. (например, имя «Нейронная сеть» будет найдено в тексте «нейронной сети»)

- *combined* объединяет обе стратегии с дедупликацией. Результаты *broad* и *strict* проходят характерный им пайплайн извлечения и обработки, после чего фрагменты дедуплицируются с помощью SHA-256 хэша и объединяются. В данном подходе результаты *strict* приоритезируются, так как к ним больше кредит доверия.

Каждая стратегия имеет свои компромиссы. Строгое извлечения имеет высокую точность, но низкую полноту - она может найти только явные упоминания.

Векторный поиск может находить косвенные упоминания, но он очень шумный. Результат его работы требует дополнительной фильтрации, что все равно не гарантирует отсутсвие ложноположительных результатов.

Комбинированный подход задуман сгладить недостатки этих стратегий путем извлечения как прямых упоминаний, так и косвенных, хоть и допуская шум.

Этап 4. Переранжирование кандидатов (Cross-Encoder). Данный этап имеет место только для векторного поиска, поскольку он зачастую возвращает большое число ложноположительных кандидатов.

Для снижения шума применяется дополнительная фильтрация кандидатов с помощью Cross Encoder. Каждая пара (чанк источника, чанк цели) подается на вход кросс-энкодеру (модель типа BAAI/bge-m3 / MiniLM). Кросс-экнодер вычисляет более точную оценку семантической связности. Пары сортируются по убыванию оценки. В финальный список попадают *top-k* пар, у каждой из которых оценка не ниже порогового значения.

Все прошедшие фильтрацию чанки группируются по файлу-цели. На их основе создаются структуры `CandidatePair`, содержащие исходный и целевой чанки, а также пути к соответствующим файлам и оценку близости из LanceDB и Cross Encoder.

Результатом этапа является список структур `CandidatePair`.

Этап 5. Классификация типа связи (LLM). Прошедшие фильтрацию пары-кандидаты передаются в языковую модель для определения точного типа семантической связи. Пары группируются по комбинации «файл-источник — файл-цель». Для каждой такой группы отбираются не более N наиболее релевантных чанков для формирования контекста на основе оценки Cross Encoder.

Логика запросов к LLM поддерживает параллелизм и батчинг. Количество одновременных запросов к модели ограничено заданным числом и контролируется семафором. Кроме того, в рамках одного запроса может проводиться классификация сразу нескольких ссылок. Это значительно снижает накладные расходы и время обработки.

Важно отметить обработку ошибок: у каждого батча есть три попытки, при израсходовании которых батч пропускается, и логгируется ошибка.

В рамках запроса модель получает системный промпт со списком допустимых типов связей и инструкциями, а также пользовательский промпт с пронумерованными парами файлов и их контекстом.

Ответ модели возвращается в формате JSON. Его парсинг производится посредством JsonOutputParser из LangChain, позволяющий восстанавливать синтаксически некорректный JSON. Также из ответов вырезается раздел с

размышлениями.

LangChain предоставляет единый интерфейс к моделям. Благодаря его использованию поддерживаются как локальные модели (например, запущенные через llama.cpp), так и облачные провайдеры (OpenAI, Google, и другие).

После каждого батча предсказания модели сохраняются в файл, формируя чекпоинт, что позволяет возобновить классификацию в случае ошибки или краша программы. Данная оптимизация призвана экономить вычислительные ресурсы, убирая потребность заново делать выполненную работу.

Этап 6. Запись связей в хранилище (ExportService). На заключительном этапе классифицированные связи сохраняются в одном из трех режимов:

- *inplace* — связи добавляются непосредственно в исходные Markdown-файлы под специальным заголовком `LINK_HEADER`. При наличии связей с предыдущих запусков новые связи недеструктивно добавятся к уже имеющимся, не создавая дубликатов.
- *json* — результаты экспортируются в JSON-файл без изменения исходных заметок. Данный режим создан для удобного переиспользования результатов, например для вычисления метрик.
- *export* — создается полная копия хранилища с вписанными связями, исходные файлы остаются нетронутыми. Этот формат сохранения полезен для эксперементов, таких как оценка GraphRAG, требующих полноценное хранилище для работы.

Формат ссылки конфигурируется (по умолчанию — синтаксис Dataview: `-relation:: [[target]]`).

Управление состоянием и отказоустойчивость (MetadataManager).

При первом запуске конвейера стандартные параметры из `config.py` сохраняются в `config.json` внутри директории `.kg_builder/`. Последующие запуски считывают этот файл и выставляют настройки, заданные в нем, что

обеспечивает воспроизводимость и повышает удобство использования на разных хранилищах с индивидуальными параметрами.

Опция `--ignore-local-config` заставляет пайплайн проигнорировать `config.json` и использовать значения из `config.py`.

Алгоритм сохраняет свое состояни с помощью создания снимка, который включает текущий этап (сканирование хранилища, сбор кандидатов, классификация, и т.д.), его прогресс и прочие метаданные, необходимые для возобновления работы.

Наибольшей детализацией обладают этапы ранжирования кандидатов и LLM-классификации отношений. У них есть логика контрольных точек, при которых сохраняется список уже обработанных файлов или пар-кандидатов. При возобновлении работы пайплайна на одном из этих шагов в алгоритм будут переданы только необработанные данные.

При последующих запусках `KnowledgeGraphBuilder` проверяет последнее сохраненное состояние. Если предыдущий запуск был прерван, то он продолжит обработку с того же шага, минуя уже выполненные.

Это сохраняет значительное количество вычислительных ресурсов и времени, особенно на дорогостоящем LLM-этапе.

Все метаданные сохраняются в отдельной директории `.kg_builder/.out/`:

- `candidates.json` — начальные кандидаты до переранжирования;
- `reranked_candidates.json` — кандидаты после переранжирования, обогащенные результатами LLM (поля `llm_type`, `llm_reasoning` дописываются после классификации)
- `predictions_partial.json` — предсказания LLM, обновляемые инкрементально
- `run_state.json` — текущее состояние алгоритма

Специальный флаг `--fresh-start` позволяет перезаписать метаданные, в том числе конфиг и список обработанных файлов. Он может быть полезен при

разработке, случаях, когда модифицируется структура метаданных, или они были повреждены.

2.2 Алгоритм GraphRAG

В качестве референсного решения был выбран ObsidianRAG [1].

Его алгоритм состоит из нескольких шагов:

1. *Индексация*: содержимое *markdown*-файлов разбивается на чанки разметом 1500 символом с перекрытием в 300 символов. Полученные фрагменты вместе с извлеченными из текста ссылками записываются в ChromaDB - векторную базу данных - которая создает для них плотные векторные эмбеддинги.

2. *Первоначальный поиск*: запрос пользователя обрабатывается с помощью `EnsembleRetriever`, использующим BM25 и ANN с весами 0.4 и 0.6 соответственно. Извлекается N наиболее релевантных заметок.

3. *Переранжирование*: полученные файлы передаются в Cross Encoder, например, `BAAI/bge-reranker-v2-m3`. В результате фильтрации остается 6 наиболее близких фрагментов.

4. *Расширение контекста*: результат дополняется пятью файлами, связанными ссылками с оставшимися.

5. *Генерация ответа*: собранное содержимое заметок передается в локальную LLM (Ollama).

У данного пайплайна есть ряд недостатков:

1. Одноэтапное расширение контекста соседями. При таком подходе теряются сложное связи, требующие несколько шагов. А также возрастает вероятность остаться в семантической ловушке: ситуации, когда новая информация не

[1] https://github.com/Vasallo94/ObsidianRAG

может быть извлечека из-за сильно разницы в эмбеддингах и ключевых словах. Суть ссылок заключается в решении этой проблемы, но автор ими пренебрегает использует.

2. Сбор контекста без учета его релевантности. В проекте уже используется Cross Encoder для первоначального поиска, его можно легко переиспользовать для фильтрации фрагметов, неотносящихся к запросу.

На основе этого решения был разработан улучшенный четырехэтапный пайплайн, поддерживающий типизированные ссылки и многошаговое расширение контекста.

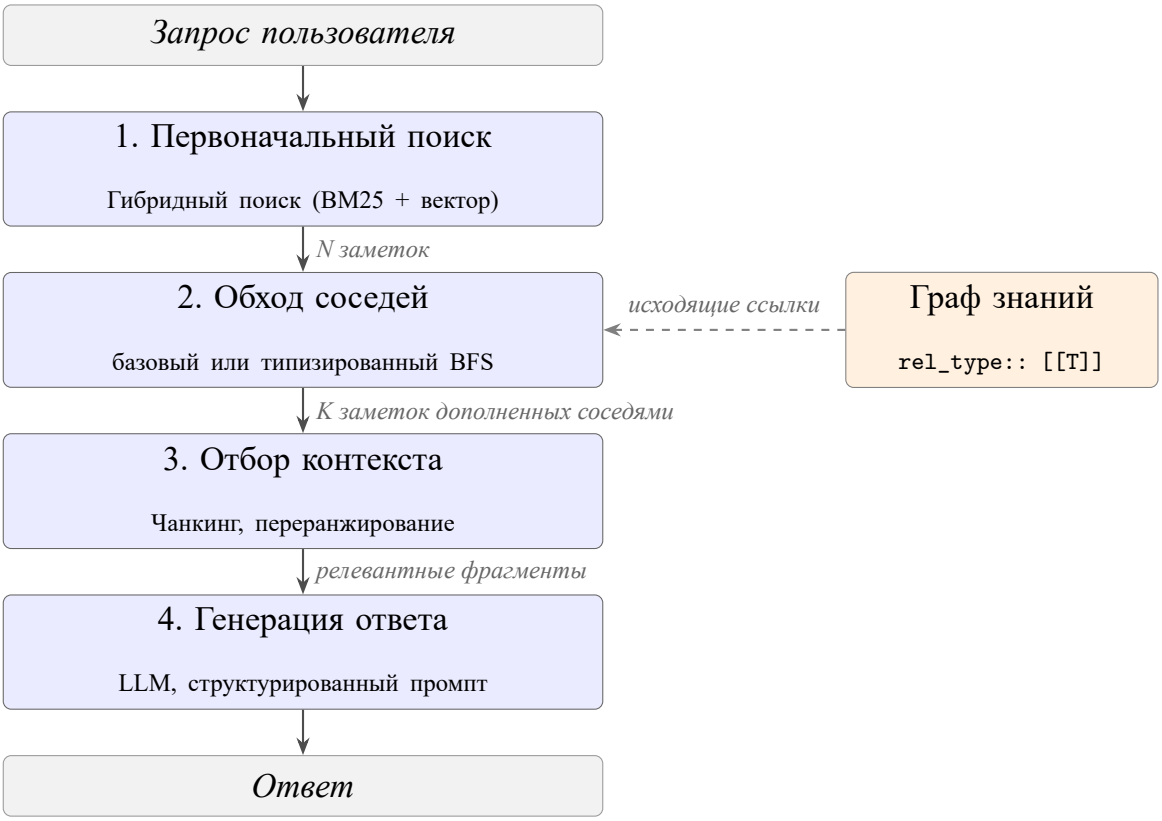

Рис. 2 – Общая схема конвейера GraphRAG

Этап 1. Первоначальный поиск. На этом этапе выполняется гибридный поиск по LanceDB. Он включает в себя ANN-поиск по плотным эмбеддингам и BM25-поиск по лемматизированному тексту. На его основе собирается k_{seed} (по умолчанию $= 5$) наиболее близких заметок-кандидатов. Они добавляются в множество посещенных файлов.

В отличие от референсного решения, на данном этапе не выполняется переранжирование заметок с помощью Cross Encoder. Оно было перенесено на шаг 3. Это решение было принято исходя из точности алгоритма извлечения, а также для смягчения критериев для отбора заметок, что потенциально может увеличить полноту собранного контекста.

Этап 2. Обход соседей.

Расширение контекста может быть проведено двумя способами:

- *Базовый обход в ширину*. Для каждой заметки формируется список исходящих ссылок. Типы ссылок при этом игнорируются, как и релевантность их содержимого. Все непосещенные файлы, соединенные с текущим, добавляются в очередь для обхода.

- *Типизированный обход в ширину*. Для каждой заметки, связанной с текущей, формируется запрос в Cross Encoder модель формата:

  `«relation: target. summary»`

  где `relation` — тип связи, `target` — имя целевой заметки, `summary` — ее краткое содержание (первый абзац). Cross Encoder вычисляет оценку близости соседней заметки с текущей. Все кандидаты с оценкой ниже пороговой не добавляются в очередь обхода. Это позволяет фильтровать заметки на этапе сбора контекста, что может снизить количество шума, но также уменьшить полноту контекста из-за раннего отброса.

Результатом этапа является множество потенциально релевантных заметок.

Наглядное сравнение стратегий обхода представлено в таблице 1.

Таблица 1 – Сравнение стратегий обхода графа

Критерий	**BaseGraphRAGPipeline**	**TypedGraphRAGPipeline**
Тип связи	Игнорируется	Включается в оценку
Оценка кандидатов	Отсутствует	Кросс-энкодер
Текст для оценки	—	`relation: target. summary`
Фильтрация	Нет (все соседи)	Пороговая по оценке
Ограничение новых узлов	Нет	Нет (порог вместо beam width)

Этап 3. Отбор содержимого. Содержимое каждой заметки, отобранной на предыдущем этапе, разбивается на фрагменты с помощью того же сплиттера, что используется при индексации хранилища.

Каждый полученный чанк получается оценку Cross Encoder схожести с запросом пользователя. Сохраняется k_{ctx} (по умолчанию 10) наиболее релевантных фрагментов, которые сформируют итоговый контекст и будут отправлены в LLM.

В отличие от референсного решения, в котором фильтрация происходит на этапе сбора контекста, здесь происходит отбор всего собранного содержимого заметок после обхода графа. Это позволяет увеличить полноту выборки, но допускает наличие в ней большего количества нерелевантных кусков.

Этап 4. Генерация ответа. Запрос в LLM формируется на основе промпта, который дает модели необходимые инструкции, например, отвечать на языке пользователя, несмотря на язык предоставленного контекста, и опираться только на предоставленные данные.

В запрос также добавляется исходный запрос пользователя и отобранные фрагменты текстов `markdown`-файлов, разграниченные специальным разделителем.

Предсказание модели ожидается в формате JSON, состоящем их двух полей: `<reasoning>` — рассуждения модели и `<answer>` — финальный ответ, возвращаемый пользователю. В случае неудачного парсинга возвращается сырой ответ.

СБОР И ПОДГОТОВКА ДАННЫХ

Для оценки качества работы пайплайна и возможности сравнения его с аналогами необходимо создать репрезентативный датасет.

3.1 Требования к данным

Перед тем как начать поиск и составление нужно сформулировать требования к данным. С учетом устройства предлагаемого конвейера (см. раздел 2.1) был создан список основных требований:

1. Исходные данные должны представлять из себя хранилища Obsidian и состоять их Markdown-файлов.

2. Корпус текстов должен быть семантически связным, то есть между файлами должны быть ярко выраженные семантические отношения (иерархические, причинно-следственные, функциональные и другие).

3. Содержание каждого файла должно быть целостным и описательным. Поскольку языковые модели нуждаются в контексте, они будут неустойчиво работать на обрывках текста или фрагментах кода.

4. Объем не должен превышать 10000 слов (ориентировочно 10-15 заметок по 600-1000 слов каждая). Данное ограничение призвано облегчить ручную разметку. В случае превышения этого порога, разметка станет крайне затруднительной, а также увеличатся риски потери связей.

5. Источник должен допускать обработку и публикацию данных.

6. Заметка должна представлять одну сущность.

Раскроем последнее ограничение подробнее. Оно означает, что в содержании файла должна находиться информация только об одной конкретной сущности, такой как личность, термин, событие, и так далее. Содержание файла может включать в себя описание этой сущности, ее определение, историю, факты, а также описание связей с другими сущностями. Такой подход делает хранилище максимально похожим на граф знаний. Кроме того, предложенная структура упрощает работу алгоритма, полькольку ему не нужно будет угадывать сущность, он будет знать ее изначально, а также не нужно будет сохранять извлеченные сущности. Этот подход отлично сочитается с философией связывания файлов в Obsidian, в которой связываются файлы целиком, а не конкретные абзацы или фрагменты текста.

На основе этого датасета, предназначенного для оценки всего алгоритма целиком, будут созданы два дополнительных для оценки извлечения контекста и классификации связей.

3.2 Поиск данных

В качестве первичной гипотезы рассматривалось использование существующих открытых пользовательских хранилищ в формате Obsidian. В ходе поиска было отобрано несколько качественных репозиториев:

- https://github.com/noodleslove/notes/ (личные заметки общего назначения)
- https://github.com/hundredrabbits/100r.co (структурированная документация коллектива)
- https://github.com/lukesmithxyz/based.cooking (корпус кулинарных рецептов)
- https://github.com/keyvanakbary/learning-notes (заметки по программной инженерии)

После их детального анализа выявился один общий критический недостаток: нарушалось правило “заметка = сущность”. Наиболее распространенными причинами его нарушения были склейки текстов или

цитат из разных источников в одном файле, это также включало файлы, преимущественно состоящие из кусков кода.

Помимо структурного несоответствия был выявлен ряд дополнительных недостатков:

- Семантическая разреженность данных. Некоторые хранилища состояли в основном из дневниковых записей, заметок или рецептов, между которыми практически не было связей, либо они были неоднозначными.
- Несоответствие объема. Почти все найденные хранилища были либо слишком малы, либо слишком велики. К репозиториям с недостающим объемом пришлось бы искусственно добавлять данные, а из слишком больших удалять, что потенциально может создать массу висячих ссылок и спровоцировать потерю ценных связей и контекста.
- В некоторых репозиториях отсутствует явная указанная лицензия либо присутствуют фрагменты текстов или кода, потенциально нарушающие авторские права. Это делает обработку этих данных рискованной.

Выбор Википедии как основного источника данных. В качестве следующего источника данных было решено использовать Википедию. У нее есть ряд важных преимуществ:

- Естественная атомарность статей. Зачастую статьи соответствуют одной сущности, что согласуется с правилом “одна заметка = одна сущность”.
- Наличие ссылок. В статьях изначально есть гиперссылки на другие статьи. Эти ссылки образуют граф, к которому можно получить доступ и использовать его как для сбора данных, так и для первоначальной разметки.
- Качество содержимого. Подавляющая часть популярных статей написана в энциклопедическом стиле, обладает высокой плотностью информации, ее последовательным и связным изложением.

3.3 Создание валидационного датасета

Построение датасета и восстановление связей. Формирование базового датасета проводилось при помощи Wikipedia API, обращение к которому производилось посредством библиотеки `wikipediaapi`. Данный инструмент позволяет сохранять тексты статей, а также предоставляет доступ к графу из ссылок. Важно отметить, что граф поддерживает типизацию ребер, что также поможет формированию датасета. Однако лишь малая его часть размечена, и в основном это ссылки, соединяющие популярные статьи.

Важно отдельно отметить, что статьи имеют специальный раздел, содержащий краткую сводку (summary). В будущем он будет использован для GraphRAG.

Формирование структуры элементов датасета. Поскольку ресурсы ограничены, необходимо определить несколько наиболее показательных сценариев для оценки алгоритма. На основе анализа слабостей GraphRAG был составлен список наиболее показательных структур будущих элеметов валидационного датасета:

- Hub-n-spoke. Структура в форме графа-звезду, состоящая из статьи-хаба и ее соседей. Главная задача алгоритма в этом случае - идентификация хаба. Алгоритмы GraphRAG зачастую пытаются обнаружить именно такие узлы, так как они дают непосредственный доступ к большому количеству информации.

- Versus. Структура, состоящая из двух разных тематик, которые местами могут показаться схожими. Например, Apple как компания и apple как яблоко. Такой подход заставит алгоритм работать с контекстов, а не только с названиями сущностей.

- Two-topics (Realistic). Данный элемент будет содержать две схожие тематики. Его задача - проверка созданных алгоритмом связей на шум.

- Sequence. Образец, состоящий из последовательных или вытекающих друг из друга событий или термиминов. Он будет нацелен на проверку способности алгоритма находить сложные, многошаговые связи.
- Personal. Элемент, созданный по аналогии с ежедневными заметками и содержащий различные события, планы, наброски идей. Он поможет оценить работу в условиях малого контекста.

3.3.1 Алгоритм генерации связей

Для формирования датасета был написан скрипт для выгрузки и обработки статей с использованием `wikipediaapi`, поддреживающий два режима сохранения:

- выгрузка заданного набора статей
- обход соседей заданной статьи в ширину с контролем глубины обхода

Во втором подходе соседи статьи ранжируются с помощью Cross Encoder модели на основе их краткой сводки и отбирается *top-n* наиболее релевантных, что предотвращает уход алгоритма от тематики корневой статьи. Далее алгоритм увеличивает глубину рекурсии и аналогично обрабатывает каждого отобранного соседа до тех пор пока либо не сохранит заданное число статей либо не обойдет все статьи до заданной глубины рекурсии.

Каждая статья, прошедная проверку Cross Encoder подвергается обработке. В нее входит:

- Нормализация названий файлов, включающая замену пробелов на подчеркивания и удаление некоторых служебных символов для однозначного соответствия названий статей названиям файлов.
- Очистка текста от сносок и маркеров (например, `[1]`), а также удаление служебных секций статьи, которые не несут содержательной информации для GraphRAG (например, “См. также”, “Литература”, “Ссылки”).

- Формирование специального блока ссылок, извлеченных из графа Википедии, на другие статьи. В данном блоке остаются ссылки только на сохраненные статьи, что решает проблему висячих ссылок.

После сохранения статей начинается этап их ручной доразметки. Этот шаг включает:

- Анализ и фильтрация ссылок, сохраненных из графа Википедии. Все ссылки, для идентификации которых нет контекста в содержимом статьи, удаляются. Например, в статье Билл Гейтс сказано, что он соосновал Microsoft, следовательно связь Билл Гейтс -> (соосновал) Microsoft должна быть оставлена.

- Анализ типов ссылок. Как было упомянуто ранее, в графе Википедии часть ссылок уже типизирована. Эти типы дополняются недостающими, а также абстрагируются для сокращения их общего числа. Критично обеспечить оптимальный уровень абстракции, чтобы не запутать языковая модель: большое количество узких по смыслу ссылок может показаться лучшим решением, но на практике модель начнет выбирать неверные типы, например из-за недостатка контекста.

Результатом ручной доразметки является блок классифицированных ссылок в каждой статье: $Extreme_ultraviolet_lithographymd\#\#RelatedConnections$: $-Mentions$ :: $[[ASML_Holding]] - Manufactures$ :: $[[Integrated_circuit]] - Sustains$ :: $[[Moores_law]] - Isa$ :: $[[Photolithography]]$

3.3.2 Алгоритм извлечения контекста

Оценку retrieval можно сформулировать как задачу воспроизведение контекстов, в которых упоминается сущность: для каждой пары (s, e), где s — файл-источник (`source_file`), а e — целевая сущность (`entity`), система должна вернуть фрагмент из текста s, в достаточной мере совпадающий с эталонным контекстом из “золотого стандарта”. Извлеченный фрагмент должен полностью

содержать эталонное предложение, а также иметь минимальное количество шума, то есть фрагментов других предложений, неупоминающих сущность.

Данный подход нацелен на точность извлечения. Очевидно, чем больше извлеченный фрагмент текста, тем выше вероятность случайного включения в него нужного предложения.

Кроме того, для LLM важен контекст, который всегда можно добавить путем объединения целевого предложения, упоминающего сущность, с некоторым количеством соседних предложений. В этом случае также важна точность извлечения, поскольку это позволит эксперементировать с добавлением контекста. Если результатом работы алгоритма retrieval будут сильно зашумленные чанки, то эти эксперименты будут сильно ограничены, возможно даже бессмесленны.

Извлеченные фрагменты текста подразделаются на две категории:

- Явное упоминане сущности. В данную группу входят все предложения, которых используется имя сущности. Например, предложение ”Билл Гейтс соосновал Microsoft” явно содержит название компании.

- Косвенное упоминание сущности. В этот класс входят предложения, в которых имя сущности заменено на местоимение. Например, в тексте ”Microsoft была основана 4 апреля 1975. Билл Гейтс был одним из тех, кто ее соосновал” местоимение ее подразумевает Microsoft.

Датасет хранится в виде набора JSON-файлов (по одному на элемент эталонного набора данных для оценки алгоритма генерации ссылок) в директории `data/test_retrieval/gold/`. Каждый файл содержит массив кортежей $(s, e, sent, direct)$, где s — файл-источник, e — целевая сущность, $sent$ — предложение, в котором упомянута сущность, $direct$ — содержит ли предложение прямое упоминание сущности.

3.3.3 Алгоритм LLM-классификации отношений

Оценка типизации связей представляет из себя задачу многоклассовой классификации отношения пары (s, e) при условии, что retrieval предоставил набор релевантных контекстов.

В качестве датасета используется дополненная версия “золотого стандарта” для оценки retrieval. В JSON-файлы было добавлено поле `link_type`, соответствующее истинному типу отношения.

Вывод.

Для оценки качества и устойчивости алгоритмам к разным сценариям использования были сформированы датасеты, различающиеся тематикой, плотностью информацией, сложностью и набором сущностей, для оценки как всего пайплайна, так и для конкретных его этапов. Данный подход задуман для упрощения тонкой настройки гиперпараметров и выборы оптимальных стратегий извлечения контекста и классификации связей.

Выбор в пользуй Википедии обеспечил выполнение правила “одна заметка = одна сущность” и позволил получить управляемый объем данных с качественным содержимым, достаточным для оценики GraphRAG.

В таблице 2 приведена информация о собранных данных.

Таблица 2 – Сводная статистика по датасетам

Подкорпус	Число заметок	Число типизированных ссылок	Число контекстных
`hub_n_spoke`	15	77	96
`realistic`	15	79	103
`versus`	15	44	62
`sequence`	7	23	44
`personal`	11	23	22

ЭКСПЕРИМЕНТЫ

4.1 Оценка конвейера построения графа знаний

4.1.1 Извлечение кандидатных пар

Извлечение может быть осуществлено тремя способами: strict, broad и combined. Подробное описание этих стартегий приведено в.

Стратегия Strict. В таблице 3 приведены результаты работы стратегии Strict на валидационном датасете для алгоритмов извлечения.

Precision	**Recall**	**F_1**
1.0	0.376	0.547

Таблица 3 – Результат Strict на валидационном датасете

Список извлеченных фрагментов не содержит ложноположительных результатов, что обеспечивает идеальную точность. Однако такой подход ограничен по полноте, так как он не способен извлечь чанки, в которых искомая сущность не упомянута явно. Таким образом, strict подход задает верхнюю границу по точности и нижнюю по полноте. Стоит отметить, что эта стартегия извлекла все прямые упоминаний сущностей, что было достигнуто путем лемматизации текстов.

Стратегия Broad.

Качество работы данной стратегии зависит от значений параметров, а также используемых моделей.

Первоначальные настройки указаны в таблице 4. Их выбор был основал на значениях, используемых в референсном решении https://github.com/Vasallo94/ObsidianRAG/tree/main а также на структуре валидационного датасета. Поскольку в нем используются markdown-файлы, имеет смысл приоритизировать разбиение текста на основе заголовков. Модели выбраны на основе их метрик и популярности на HuggingFace.

Для поиска оптимальной конфигурации был проведен ряд экспериментов, заключающихся в смене сплиттера, размера чанка, моделей эмбеддингов и переранжирования. Их результаты представлены в таблицах 5 и 6.

Параметр	**Значение**
Эмбеддинги и разбиение текста (Chunking)	
Модель эмбеддингов	`paraphrase-multilingual-MiniLM-L12-v2`
Размер чанка	3000
Перекрытие чанков	300
Разделители	`["\n# ", "\n## ", "\n### ", "\n\n", ". ", "? ", "! ", "\n", " ", ""]`
Оконная стратегия (Sentence Window)	
Тип сплиттера	`sentence_window`
Окно до	1
Окно после	1
Поиск (Retrieval)	
Top-K	15
Вес векторного поиска	0.5
Режим запроса	`title_summary`
Реранжирование (Reranking)	
Модель реранкера	`BAAI/bge-reranker-v2-m3`
Top-K	5
Порог	0.7

Таблица 4 – Изначальные гиперпараметры алгоритма извлечения

#	Splitter	QM	K	P	R	F_1
1	sent_window	TS	15	0.226	0.043	0.072
2	sent_window	TS	30	0.258	0.070	0.111
3	sent_window	TS	50	0.217	0.070	0.106
Strict (базис)				1.0	0.376	0.547

Таблица 5 – Анализ конфигурации splitter в стратегии Broad

Также была попытка эксперимента с почанковой стратегией поиска фрагментов из других текстов, но в сочетании с текущим splitter формирование кандидатов заняло около 5 часов, что считается неприемлимым.

В последующих экспериментах будет использоваться рекурсивный разделитель текста вместе с chunk стратегией поиска фрагментов. Наложение чанков будет зафиксировано равным 300 символам.

#	CS	E	K	CE	Порог	P	R	F_1
4	3000	MiniLM	15	bge-reranker	0.7	0.395	0.190	0.256
5	5000	MiniLM	15	bge-reranker	0.7	0.301	0.186	0.223
6	3000	MiniLM	15	mMiniLMv2	0.7	0.338	0.407	0.369
7	1000	MiniLM	15	mMiniLMv2	0.7	0.270	0.360	0.309
8	5000	MiniLM	15	mMiniLMv2	0.7	0.309	0.339	0.324
9	1000	MiniLM	15	jina	0.7	0.450	0.150	0.225
10	3000	MiniLM	15	jina	0.7	0.553	0.208	0.302
11	3000	MiniLM	15	jina	0.5	0.510	0.407	0.452
12	3000	MiniLM	30	jina	0.5	0.500	0.453	0.475
13	3000	bge-m3	30	jina	0.5	0.531	0.468	0.498
Strict (базис)						1.0	0.376	0.547

Таблица 6 – Анализ конфигураций стратегии Broad. CS - Chunk Size, CE - Cross Encoder, E - Embedding

Ключевые выводы.

Влияние сплиттера и размера чанка. Эксперименты 1-3, 7, 9 показалы, что фрагмент текста не может быть слишком маленьким, так как это приводит к потере контекста. Эксперименты 5 и 8 дают понять, что слишком большой размет также негативно влияет на качество извлечения из-за потери деталей и усреднении смыслов при переводе в эмбеддинговое пространство.

Самый большой рост метрик обеспечил переход от оконного разбиения к рекурсивному. Стратегия оконного разбиения обладает крайне высоким перекрытием чанков: каждое предложение попадает в три чанка. Кроме того, полученные фрагменты не содержат достаточно контекста.

Большое количество опытов с размером фрагментов обусловлено отсутствием референсных значений: пришлось вслепую искать оптимальное значение.

Роль модели реранжирования. Модель Cross Encoder также оказалась крайне важной. Смена bge-reranker на mMiniLMv2 дала прирост $\Delta F_1 = +0.113$. Переход с mMiniLMv2 на jina увеличил F_1 до 0.452, но потребовал снижения порога с 0.7 до 0.5 для раскрытия потенциала модели. Изменение порога является естественным решением при смене архитектуры: некоторые модели строже, из-за отсекают часть релевантных кандидатов.

Количество извлеченных чанков. Параметр Top-K для извлечения K наиболее близких фрагментов оставался неизменным до 12 эксперимента, так как увеличение его значения спровоцировало был рост шума, что вместе со слабыми моделями могло негативно повлиять на точность, а снижение отбросило бы потенциально нужные чанки. Решение о его увеличении было принято только после достижения удовлетворительной точности, и, как ожидалось, из-за него упала точность и выросла полнота.

Смена модели эмбеддингов. На замену MiniLM, переводящую тексты в 384-мерное пространство пришла более тяжелая bge-m3, которая работает с 1024-мерными эмбеддингами. Идея была в улучшении сохранения детализации

смыслов в текстах. Как итог, рост точности и полноты на 0.031 и 0.015 соответственно.

Стратегия Combined.

В данной стратегии использовалась оптимальная конфигурация Broad.

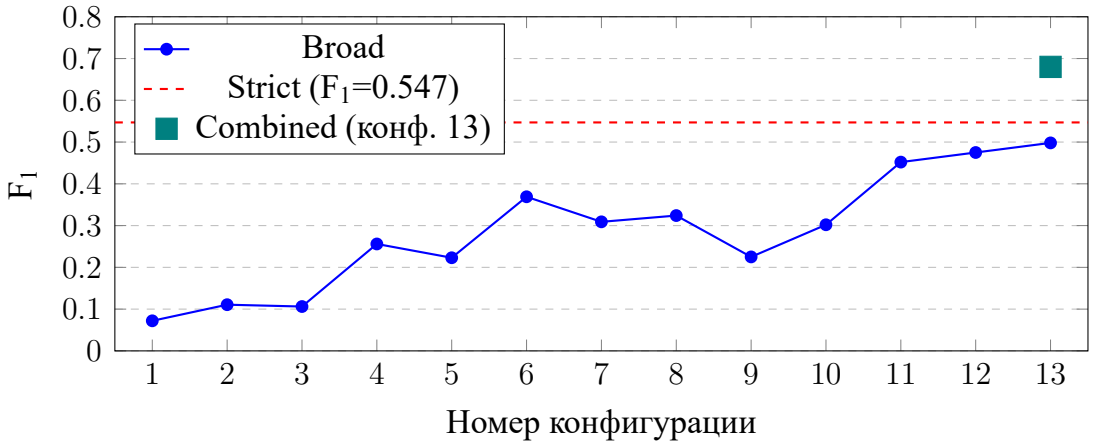

Рис. 3 – Динамика F_1 стратегии Broad по конфигурациям. Красная пунктирная линия — уровень Strict-стратегии. Квадрат — результат Combined для лучшей конфигурации.

4.1.2 Классификация типов связей

Модель	Промпт	Batch	P	H	R	S	V	Overall
gemma-3	base	1	0.346	0.340	0.351	0.459	0.522	0.393
gemma-3	base	5	0.446	0.325	0.321	0.451	0.573	0.387
gemma-3	base	10	0.378	0.428	0.363	0.392	0.572	0.423
qwen-3	base	1	0.313	0.454	0.272	0.454	0.526	0.392
gemma-3	few-shot	1	0.372	0.436	0.444	0.341	0.559	0.445
qwen-3	few-shot	1	0.498	0.390	0.434	0.558	0.482	0.445
gemma-4	few-shot	1	0.856	0.578	0.627	0.786	0.758	**0.669**

Таблица 7 – Weighted F_1 классификации типов связей

Влияние батчинга. Увеличение размера батча снижает среднюю оценку, однако в некоторых случаях она растет. Увеличение может быть вызвано передачей дополнительного контекста для классификации других ссылок в батче. Это видно в случае Personal, Hub-n-Spoke и Versus, где размер содержимого заметок мал, и есть кластеры и тематически связных. Тем не менее, важно не перегрузить модель лишним контекстом: при классификации конкретной связи контекст для других считается шумом и не должен быть учтен. Не все модели могут корректно разграничить содержимое, из-за чего начинают путаться и делать ошибочные выводы, ухудшая качество классификации.

Как наиболее безопасный компромисс был выбран размер батча равный 1.

Влияние промпта. Добавление в промпт примеров и описаний типов связей, которые в них используются увеличило значение Overall F_1 как для Gemma-3-27b с 0.392 до 0.423 (+0.031), так и для Qwen-3-235b с 0.392 до 0.445 (+0.135). Модели чувствительны к инструкциям и примерам, они помогают им лучше понять специфику задачи и не уйти в сторону при рассуждении. Особенно важно точно определить каждый тип связи, чтобы устранить неоднозначности. Результаты показывают, что качество более крупных моделей больше зависит от промпта.

Влияние выбора модели. Более современные модели с эффективными архитектурами превозходят старые, даже имея значетельно меньше параметров. Новая Gemma 4 имеет всего 31 миллиард параметров, при этом она на голову выше Qwen с 235 миллиардами параметров.

Анализ ошибок классификации.

Все конфигурации имеют ряд классов с низким F_1:

- *Tun* `mentions` (40 связей, F_1 = 0.23–0.39). Это самый популярный тип связи в датасете. Он играет роль fallback: связь получает этот тип в случае, когда невозможно однозначно назначить ей другой класс. Модель может злоупотреблять этим типом, так как практически во всех случаях он уместен: сущность действительно упомянута. Эта проблема решается

дополнительными инструкциями и примерами в промпте, чтобы явно указать смысл этой связи и когда ее уместно использовать.

- *Близкие по смыслу связи*, такие как `encompasses`, `contains` и `incorporates`. Большое их количество создает путаницу: модель не видит особой семантической разницы между ними, поэтому с высокой вероятностью может использовать неверный тип. Проблема обостряется в реальных условиях неполного контекста с шумом, который создает алгоритм извлечения контекста. Эту проблему можно устранить объединением подобных классов в один более универсальный.

4.2 Оценка конвейера GraphRAG

Оценка GraphRAG-конвейеров проводилась с помощью фреймворка RAGAS, который полностью автоматизирует процесс. Были выбраны четыре метрики:

- **Context Precision (CP)** – доля контекстных фрагментов, использованных для формирования правильного ответа. Измеряет точность поиска: низкое значение указывает на большое количество нерелевантных чанков.
- **Context Recall (CR)** – доля эталонных фактов, присутствующих в извлеченном контексте. Измеряет полноту поиска: значение 1.0 означает, что весь необходимый для ответа материал был найден.
- **Faithfulness (F)** – степень фактологического соответствия ответа предоставленному контексту. Значение, близкое к 1.0, означает, что модель не галлюцинирует.
- **Answer Correctness (AC)** – метрика близости сгенерированного ответа к эталонному. Учитывает как семантическое сходство, так и точность фактов. Эта метрика является главным критерием общего качества системы.

4.2.1 Референсное решение (ObsidianRAG)

В качетве модели для генерации ответов была выбрана Gemma-3-27b, моделью для оценка для оценки была Gemma-4-31b.

Подкорпус	**CR**	**CP**	**F**	**AC**
`hub_n_spoke`	0.802	0.701	0.955	0.542
`personal`	0.625	0.615	1.000	0.424
`realistic`	0.886	0.707	0.898	0.505
`sequence`	1.000	0.824	0.967	0.582
`versus`	0.617	0.467	0.883	0.428
Среднее	0.786	0.663	0.941	0.496

Таблица 8 – Результаты референсного решения ObsidianRAG

Результаты оценки референсного решения приведены в таблице 8. Явно видны недостатки логики расширения контекста: в `personal` и `versus` наблюдается низкий CR (0.625 и 0.617 соответственно). В этих образцах есть два кластера, которые практически друг с другом не связаны. Банальный сбор контекста соседей не позволит получить информацию из другого кластера, а лишь увеличит шум, что видно по низким значениям CP (0.467 в случае versus и 0.615 в случае personal).

В других сэмплах, где заметки связаны большим числом ссылок, CR не ниже 0.8, а CP не ниже 0.7, что говорит о хорошей работе алгоритма.

Во всех случаях наблюдается высокий Answer Faithfulness: модель не пытается додумать ответ, а использует приведенные данные. Из этого следует и низкий AC: у модели попросту нет необходимых данных для предоставления полного и точного ответа.

Датасет	Context Recall			Answer Correctness		
	Ref	**Base**	**Typed**	**Ref**	**Base**	**Typed**
`hub`	0.802	**1.000**	0.958	0.542	**0.587**	0.565
`personal`	0.625	**1.000**	**1.000**	0.424	**0.597**	0.579
`realistic`	0.886	**0.977**	**0.977**	0.505	**0.606**	0.601
`sequence`	**1.000**	0.925	0.975	0.582	0.580	**0.631**
`versus`	0.617	**0.900**	0.788	0.428	**0.524**	0.507

Таблица 9 – Сравнение полноты извлечения (CR) и точности ответа (AC) между реализацией ObsidianRAG и собственными алгоритмами.

Поскольку основным преимуществом алгоритмов, представленных в данной работе, является многошаговый обход, основная цель которого увеличение CR, было проведено их сравнение с эталонным решением, которое представлено в таблице 9.

Видно, что модифицированная логика обхода действительно позволила алгоритму собирать больше релевантной информации, позволив получить практически идеальную полноту контекста. Это оказалось особенно полезно в случае `personal` и `versus`, где изначально значение CR было самым низким. В их случае прирост составил 0.375 и 0.283 соответственно в случае Base и 0.375 и 0.171 в случае Typed. Дополнительный контекст помог языковой модели дать более точный ответ, что также увеличило значения AC.

4.2.2 Влияние промпта и языковой модели

Для исследования влияния различных факторов и компонент GraphRAG на качество ответа LLM была проведена серия экспериментов над решениями, представленными в данной работе.

Базовый промпт против few-shot. На основе экспериментов над LLM-классификацией связей был сделан вывод о важности указания точный инструкций и конкретных примеров в промпте. В базовом промпте уже есть

основные указания, такие как придерживание предоставленного контекста и ответ на языке запроса. К ним будут добавлены конкретные развернутые примеры и требования модели к рассуждениям перед ответом. Модели для оценки и генерации остались теми же.

Алгоритм	**Промпт**	H	P	R	S	V	**AC avg**
Base	Базовый	0.587	0.597	0.606	0.580	0.524	0.579
	Few-shot	0.595	0.572	0.604	0.618	0.569	0.592
Typed	Базовый	0.565	0.579	0.601	0.631	0.507	0.577
	Few-shot	0.600	0.611	0.606	0.629	0.555	0.580

Таблица 10 – Сравнение базового и few-shot промптов на алгоритмах Base и Typed (метрика AC, модель Gemma-3-27b)

Новый промпт продемонстрировал прирост AC в среднем лишь +0.13. Прирост оказался ниже, чем в предыдущих сравнениях, вероятно, из-за корретно сформулированных инструкций в базовом промте. Можно сделать вывод, что наличие примеров все же дает небольшой прирост корректности ответа языковой модели.

Анализ влияния масштаба языковой модели.

Модель для генерации будет заменена на Gemma-4-31b, а моделью-судьей будет Gemini-3-flash.

Датасет	Алгоритм	Эксп. 2 (Gemma-3-27b)			Эксп. 3 (Gemma-4-31b)		
		CP	CR	AC	CP	CR	AC
`hub`	Base	0.749	1.000	0.595	0.660 ↓	0.875 ↓	0.698 ↑
	Typed	0.796	0.958	0.600	0.707 ↓	0.833 ↓	0.709 ↑
`personal`	Base	0.893	1.000	0.572	0.750 ↓	1.000 =	0.796 ↑
	Typed	0.892	1.000	0.611	0.729 ↓	0.917 ↓	0.770 ↑
`realistic`	Base	0.848	0.977	0.604	0.426 ↓	1.000 ↑	0.738 ↑
	Typed	0.851	0.977	0.606	0.491 ↓	0.955 ↓	0.792 ↑
`sequence`	Base	0.879	0.925	0.618	0.434 ↓	0.765 ↓	0.734 ↑
	Typed	0.878	0.975	0.629	0.429 ↓	0.765 ↓	0.706 ↑
`versus`	Base	0.734	0.900	0.569	0.519 ↓	0.655 ↓	0.623 ↑
	Typed	0.696	0.813	0.555	0.547 ↓	0.568 ↓	0.576 ↑

Таблица 11 – Сравнение моделей Gemma-3 и Gemma-4 (few-shot промпт)

Прирост AC составил в среднем +0.128, при этом CP упал в среднем на -0.263. Сильнее всего CP снизился на `realistic` (с 0.848 до 0.426) и `sequence` (с 0.879 до 0.434). Объяснить это можно тем, что модель иначе формирует ответ, используя все чанки из контекста, в том числе ложноположительные, что снижает метрику, но при этом повышает AC.

Сравнение эталонных ссылок и сгенерированных.

Датасет	Алгоритм	Эксп. 3			Эксп. 5		
		CP	CR	AC	CP	CR	AC
`hub`	Base	0.660	0.875	0.698	0.689 ↑	0.833 ↓	0.682 ↓
	Typed	0.707	0.833	0.709	0.707 =	0.833 =	0.716 ↑
`personal`	Base	0.750	1.000	0.796	0.729 ↓	0.958 ↓	0.762 ↓
	Typed	0.729	0.917	0.770	0.729 =	0.917 =	0.770 =
`realistic`	Base	0.426	1.000	0.738	0.426 =	1.000 =	0.738 =
	Typed	0.491	0.954	0.792	0.476 ↓	0.977 ↑	0.774 ↓
`sequence`	Base	0.434	0.765	0.734	0.434 =	0.765 =	0.734 =
	Typed	0.429	0.765	0.706	0.429 =	0.765 =	0.706 =
`versus`	Base	0.519	0.655	0.623	0.519 =	0.655 =	0.623 =
	Typed	0.547	0.568	0.576	0.547 =	0.568 =	0.576 =

Таблица 12 – Сравнение качества работы GraphRAG между размеченным графом и сгенерированным.

У `sequence` и `versus` значения метрик остались неизменными. В всех других случаях метрики изменились незначительно, причем в основном они снизились.

Разница в Answer Correctness между реальными и сгенерированными ссылками не превышает 0.18 для Base-обхода и столько же для Typed-обхода в среднем. Кроме того, значения CP и CR также отличаются минимально, что говорит о высокой устойчивости к шуму алгоритма генерации ссылок.

Сравнение типизированного обхода с обычным.

Предложенные в этой работе улучшения алгоритма GraphRAG для использования в Obsidian показали свою действенность, увеличив полноту контекста и, как следствие, его корректность.

У типизированного обхода выше CP (среднее: 0.581 против 0.558 у Base), но ниже CR (0.808 против 0.859 у Base). Эти результаты показывают влияние

фильтрации заметок при обходе графа: тематически нерелевантные файлы отсекаются, но при этом также теряется возможность сделать дополнительный шаг через этот файл, чтобы оценить его соседей, из-за чего упускается часть информации.

4.3 Итоговое сравнение и ограничения исследования

Важно отметить, что во всех экспериментах значение метрики Answer Faithfulness было не ниже 0.88, что говорит об отсутствии галлюцинаций и использовании языковыми моделями приведенного контекста.

ЗАКЛЮЧЕНИЕ

СПИСОК ЛИТЕРАТУРЫ

ПРИЛОЖЕНИЕ

Репозиторий с кодом: https://github.com/conk7/knowledge_graph_builder